

Prof Dr rer nat habil
Tobias Weinzierl (FHEA)
Institute for Data Science
Department of Computer Science
Durham University
DH13LE Durham
GREAT BRITAIN

29 August 2026

To whom it may concern,

Re: Evaluation of the thesis of Mr. Jiri Klepl

The thesis addresses the fact that the success of modern HPC implementations hinges upon an efficient implementation: the realisation of a given algorithm has to squeeze as much out of the hardware as possible. The candidate addresses this challenge through two strands: First, the design of high-performance, portable abstractions for CPU and GPU architectures, centered on two design principles: layout-agnostic and traversal-agnostic designs (there's also parallelism-agnostic design in the thesis, which could be read as third principle). Realised through C++ meta-/generic programming, the candidate suggests that programmers can write down implementations in an almost machine-agnostic way, and that we can completely separate the tuning of the memory layout and traversal from this and even hide it away. A second strand of the thesis explores how large language models (LLMs) can be brought into this optimisation space, i.e. whether they can exploit this two-tier abstraction as developed and perform optimisations on top of it. The thesis comprises five main chapters plus an appended set of 11 candidate-authored papers.

Below I go through the thesis chapter by chapter, followed by comments on the literature base as well as an overall assessment.

Chapter 1

The chapter opens by stating that algorithm performance on modern HPC architectures depends on factors such as machine topology (GPU and CPU integration), memory arrangement, and task scheduling, before narrowing the focus to memory access patterns. This framing makes sense in the context of the present thesis, yet is not fully convincing as written:

implementation choices do not merely follow algorithmic decisions — increasingly, optimising code requires developers to go all the way back to the design of their original code. For example, on some systems it might be reasonable to exploit sparsity of linear equation systems, while others favour treating any system as dense. In a certain sense, the motivation for privileging memory access patterns over the others deserves more justification than is given, but given that the chapter sets the tone for the remainder, the choice is narrow yet still valid.

The observation that "optimizing data access patterns is important primarily because modern HPC architectures have specialized hardware features that improve the performance of specific access patterns" is fundamentally different from a lot of hardware-agnostic optimisation approaches as we find them in computer science. It is an interesting motivation behind the thesis, which could even have been made stronger. For a scientific thesis, the candidate could have discussed hardware-oblivious solutions and compared them explicitly to his train of thought. Next, the candidate presents the second strand of the work: the idea that LLMs could step into this space (i.e., learn or predict which access patterns suit which hardware). Again, this is a reasonable one, but the reader would have benefited from being told this more explicitly.

Besides laying out the narrative, the chapter also introduces the underlying hardware description. While overall sound, the claim that data transfers are always necessary on GPUs is too strong. Offload-based transfers remain the dominant model, but memory-persistent approaches (where data resides on the GPU across kernel invocations) exist and should be acknowledged, as well as architectures offering virtual shared memory (superchips).

The CPU discussion runs through cache usage, vectorization, and parallelisation as key performance ingredients. The GPU discussion introduces parallelism as arising from sufficient concurrency, but it is unclear why this same framing and the logical link to SIMT are not picked up in the CPU discussion. One can read GPUs as massively SIMT architectures, while CPUs offer fewer cores but complex multi-socket architectures — enough that it is surprising concurrency is not discussed for CPUs, too. This feels inconsistent.

The candidate makes a point on overlapping data transfer and compute, which is a genuinely interesting contribution to the chapter: data layout and traversal order decisions inherently constrain or offer opportunities to hide data transfers behind actual work.

Finally, the author touches upon compilers and templates, which later turn out to be a key tool used: the introduction of C++ templates as "abstraction" is defensible (templates are one realisation of abstraction), but the claim that "compilers can interpret these abstractions and optimize them effectively" is unclear in the templates context. Mainstream compilers do not, in fact, "read" templates in the sense implied. To the best of the reviewer's knowledge, templates are instantiated and then optimised. The compiler does not offer bespoke optimisation passes for the template itself. What is true, however, is that templates allow programmers to inject compile-time evaluation into their code base. This observation is correct. But it remains unclear whether the candidate wants to imply this or something else.

Chapter 2

The chapter surveys different mechanisms for exposing optimisations to users (DSLs, annotations, native abstractions).

Section 2.1.1 is introduced without a corresponding 2.1.2, which is a structural oversight. The terminology used around DSLs would benefit from a stronger literature grounding. The distinctions drawn are sensible in themselves, but they diverge from the terminology predominant in the field, where the more common distinction is between embedded and external DSLs. The field does not have a fixed de facto standard of terms. It would hence be all the more important to reconcile and compare the chosen terminology with the existing literature.

The distinction between algorithm specification and schedule is well made and useful.

The distinction between user-knowledge-driven and heuristics-driven approaches deserves more emphasis than it currently receives, notably as the candidate navigates between those two camps, i.e. first introduces mechanisms that allow users to optimise code and then turns to LLMs to do exactly this.

The claim that "while annotation-based approaches are prominent in the context of parallelization (especially OpenMP pragmas), they do not offer significant characteristics that would not be covered by DSLs" is questionable — arguably OpenMP itself has DSL-like characteristics, which complicates this framing and should be addressed. The statement that "the design space for efficient native abstractions is more limited than for the above alternatives" needs supporting evidence; it currently reads as an unsupported assertion. That said, the contextualisation of Kokkos et al. within this space is a nice addition. At later points, the thesis would benefit from a stronger comparison to these quasi-industry standards. Here, it is excellent.

Nevertheless, it is unclear how C++ templates relate to the claim about data layouts that "cannot be achieved otherwise." Templates provide type-generic implementations and static evaluation, but the connection to data layout specifically is not established. The statement becomes reasonable only once data layout is explicitly tied to types — but that link is not made in the text. Overall, some of the statements in this context might be correct as long as the candidate refrains from considering compiler modifications and extensions themselves. The "template-exclusive" statements are a weak point of the chapter and need to be defended.

The autotuning discussion would benefit from a distinction between static and on-the-fly (dynamic) autotuning, along with a brief overview of the feedback quantities typically used. Runtime is treated as the dominant tuning signal, but it is not the only one — once other quantities (e.g., memory access metrics) are incorporated, tuners generally perform better, and this point is currently underdeveloped.

The discussion of expert heuristics is a good addition.

The LLM discussion is adequate; it correctly notes that LLMs can "identify," or more accurately predict, execution patterns, and that this gives them an advantage over other heuristic models, which cannot predict or "explain" the impact of their transformations. This point is, in effect, the core motivating idea of the second part of the thesis as a whole and could be foregrounded more explicitly as such.

Chapter 3

This chapter hosts the core contributions of the work. Despite this, it is very short. The reason is that much of it becomes a forward reference to peer-reviewed papers written by the candidate.

As a start, the candidate introduces the core idea of adding an indirection to facilitate memory layout-agnostic programming: an additional mapping from a logical data arrangement to a physical space. Instead of accessing an array element directly, for example, an intermediate object takes the array index and maps it onto a real position, which allows this intermediate object to permute data totally hidden from the user's implementation. Traversal-agnostic design pushes this idea further by specifying iteration spaces rather than explicit loops. Now it is up to the realisation of the index space to yield different index orderings. Such an iteration abstraction can, naturally, subsume index permutations (i.e., indirections) as a special case.

These ideas are then exercised by means of five contributions, which are really external contributions: the candidate very briefly sketches these peer-reviewed pieces of work and then adds some outlook and remarks on future work.

Some of these discussions suffer from structural inconsistencies. For example, 3.6.1 comes along without a 3.6.2. Overall, such a treatment of previously published contributions in the tradition of cumulative theses is problematic in two related ways:

On the one hand, the explanations are insufficiently self-contained. The thesis does not provide enough detail about what was actually done in the referenced work, making the follow-up text very hard to follow without having read the original papers. At minimum, a longer summary and sketch of the key ideas of each contribution would have helped the reader.

On the other hand, the text suffers from overstated novelty. The text is written in a way that risks giving the impression that the candidate invented the two optimisation paradigms (layout-agnostic and traversal-agnostic design) outright. This is misleading and, notably, does not serve the candidate's own case well: the candidate's contribution here is best characterised as a minor, incremental one within an existing landscape of optimisations. Which of these ideas can also be found in software like Kokkos or OpenMP's loop transformations?

Yet, correcting this framing, i.e. weakening the statements, would strengthen the thesis's later argument, since it implies that the subsequent LLM-based work is applicable to a wide range of existing approaches by other authors — not solely to the candidate's own core contribution.

Chapter 4

Chapter 4 continues in the tradition of Chapter 3 and presents the core research work again as an extended reference to the candidate's prior publications. Section 4.3 is probably the most interesting part of it. It observes that prompt quality plays a key role in LLM performance (unsurprising), but more strikingly, that abstractions designed for human comprehension can become genuine challenges for LLMs to work with. This is a surprising and valuable observation.

The main limitation here is that the candidate does not provide a reasoned explanation for this phenomenon — the observation is made but not really analysed. A natural direction for future work (worth stating explicitly) is agent-based follow-up research that incorporates feedback loops, e.g., using actual performance measurements as a signal.

Chapter 5

The last chapter (minus a conclusion) describes the two major software components developed as part of the thesis. As is often the case with software package chapters, this section is not, in itself, the most intellectually interesting part of the thesis (the more interesting results are in Chapters 3 and 4), but it serves an important purpose: enabling comparison of the candidate's implementations against other approaches in the field.

This comparison, however, is not actually carried out by the candidate — the candidate restricts the discussion to a description of the home-grown solution. It would be both timely and relevant to compare the two software realisations against established solutions such as Kokkos, explicitly highlighting both the added value and the weaknesses relative to such alternatives. This is a notable gap.

Chapter 6

The thesis concludes: "We have presented the concept of layout-agnostic and traversal-agnostic design and shown that these design principles can be applied to high-performance computing via abstractions that rely solely on the metaprogramming capabilities of C++." This captures one strand of the thesis clearly. The transition into the LLM strand of the work in the remainder of the summary could be made more explicit and better integrated with this first strand.

Literature and structure

The candidate has authored 11 papers, several in well-regarded venues, while others are still under review. These papers are appended to the thesis. The actual literature list is relatively short, with 64 references in total, of which 11 are the candidate's own papers. A significant number of the remaining references are preprints, technical reports, or software whitepapers rather than peer-reviewed literature. Overall, the literature presented supports the observation that the presentation is in places very self-centric and does not invest much effort into comparing ideas to the wider trends in the field.

The main text itself is very short (36 pages in total), which is due to the cumulative nature of the presentation.

Overall Assessment

The thesis maps out an interesting research direction in that it connects layout/traversal-agnostic abstraction design with LLM-based optimisation. This is genuinely creative research. Together with the sound number of (published) papers, it provides evidence that the candidate is mature enough to conduct their own, independent research. Section 4.3's observation about human-oriented abstractions posing challenges for LLMs is a particularly valuable and non-obvious finding.

The presentation of the thesis has flaws. There are structural/terminological inconsistencies (e.g. between the CPU and GPU discussions in Chapter 1), and the reader finds several unsupported or imprecise claims (e.g., GPU data transfers, compiler interpretation of templates, native abstraction design space). These would benefit from tightening. Appending papers without summarising key ideas in detail in the main text makes reading the work challenging.

Probably the biggest flaw of the thesis is the lack of grounding in related research (cmp. limited literature and lack of conceptual comparison to other work), culminating in some overstatements: "In our research towards enabling developers to easily design high-performance algorithms for parallel computing on CPU and GPU architectures, we have introduced the concepts of layout-agnostic and traversal-agnostic design, which had not been extensively studied prior to our work. Yet, many works in the area of high-performance computing have incidentally applied some of the principles." Without evidence, such a statement is very problematic, notably as the candidate then does not compare their solution to others in detail.

Scientifically speaking, we observe that two directions of research are neglected which would have pushed the boundary of knowledge significantly: (a) There needs to be an investigation into why the LLM-based approach struggles. This is counterintuitive and might either suggest that the way data is fed into the LLM is inappropriate, or that there is a fundamental workflow issue (e.g. only providing reinforcement which misguides the optimiser). (b) While understandable (albeit not clarified that explicitly), the author constrains themselves to C++ templates. Today, a lot of effort in the community is invested in the compiler infrastructure

itself. Notably, the MLIR initiative comes to mind. It is an interesting question to what degree these approaches are substantially more powerful than C++ template realisations — or whether they do not add additional opportunities.

Given that the candidate has demonstrated that they master the principles of scientific working, this might be an interesting future direction of research. But even without exploring it, the thesis makes a contribution to the field. I can recommend the thesis for defense.

With regards,

Tobias Weinzierl
Durham University